Step 1 — A Classifier That Sends Each Ticket to the Right Reply Chain — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 1: every later step makes one part of this pipeline stronger — faster lookups, safer failures, a live reply, a record. Before any of that, prove the basic shape works: one chain decides the category, and a second chain, picked by that category, writes the reply. **If not:** when Step 3's fallback fires, you won't know whether the problem is the fallback or the routing under it.

After this step the project runs: `python main.py` classifies the 4 sample tickets and prints a reply for each.

The shared files

The same files as Project 1 — Doc01's config, errors and logger, and Project 1's prompt loader — in the `src/` package, with the prompts folder `src/prompts`.

Story — `exceptions.py`, `config.py`, `utils/logger.py`, `utils/prompts.py`: settings load and fail loudly at startup, every file logs the same way, and prompts live in `.txt` files. **If not:** a missing key would surface deep inside a chain as a confusing error.

```
# src/exceptions.py
# The project's named errors. MissingConfigError: a required setting is
# missing from .env. (Doc01's file.)
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

```
# src/config.py
# Loads the settings from .env into a checked Config object. load_config()
# raises MissingConfigError when OPENAI_API_KEY is missing. (Doc01's file.)
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
```

```
class Config:
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

```
# src/utils/logger.py
# get_logger(name): one logging setup for every file — a timestamp, the
# file name and the level on every line. (Doc01's file.)
import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)

    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)

        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        formatter = logging.Formatter(log_format)
        handler.setFormatter(formatter)

        logger.addHandler(handler)
        logger.setLevel(level)

    return logger
```

```
# src/utils/prompts.py
# load_prompt(name): reads src/prompts/<name>.txt, so prompts live in text
# files instead of Python strings. (Project 1's file.)
```

```
def load_prompt(name: str) -> str:
    path = "src/prompts" + "/" + name + ".txt"
    with open(path) as prompt_file:
        return prompt_file.read().strip()
```

Also create empty `__init__.py` files in `src/utils/` and `src/models/`.

> Back to this step · Hint 1 · Hint 2 · Solution

Solution — schema, chains, prompts and a service, each in its own file

Story — `models/llm.py`: one place builds the chat model, so the model choice lives in one line, never scattered (the rule from Doc05's Build Task). **If not:** every chain would build its own `ChatOpenAI` with its own settings.

```
# src/models/llm.py
# get_model(): the one ChatOpenAI every chain uses. ChatOpenAI reads
# OPENAI_API_KEY from the environment, filled by load_config().
from langchain_openai import ChatOpenAI

def get_model() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

Story — `schemas.py`: the classifier's answer is a contract: `category` can only be one of three values, `reason` explains the choice, and `sure` says whether the ticket could fit more than one category. `TicketRecord` is the result of one ticket — customer, ticket, category and reply — checked when it's built. Later steps only **add** fields to it, each with a default. **If not:** a typo like `"Billing"` would silently match no branch, and every caller would guess the keys of a loose dict.

```
# src/schemas.py
# The pipeline's checked shapes. TicketCategory is the classifier's answer
# (only 3 categories allowed); TicketRecord is one ticket's result. Later
# steps add fields to TicketRecord, each with a default.
from typing import Literal

from pydantic import BaseModel

class TicketCategory(BaseModel):
    category: Literal["billing", "technical", "general"]
    reason: str
    sure: bool

class TicketRecord(BaseModel):
    customer_id: str
    ticket: str
```

```
    category: str
    reply: str
```

Story — `prompts/classifier.txt` : the rules for sorting, in plain text. `{ticket}` is the one slot the template fills. **If not:** rewording the classifier would mean editing Python.

`src/prompts/classifier.txt`

```
You sort customer support tickets into exactly one category:

- billing: charges, invoices, refunds, payments, plans
- technical: something in the product is broken or not working
- general: anything else, like questions about the company

Give a one-sentence reason. Set sure to false if the ticket could
fit more than one category.

Ticket: {ticket}
```

Story — the three reply prompts: one per category, named exactly like the category. Each has its own rules — billing never promises a refund, technical asks for a missing detail, general never invents a company fact. **If not:** one prompt trying to cover all three would do each one worse.

`src/prompts/billing.txt`

```
You are a billing support agent at SupportCo. Write a short, polite
reply to the customer, under 120 words. Never promise a refund: say
the billing team will check the charge within 2 business days.

Ticket: {ticket}
```

`src/prompts/technical.txt`

```
You are a technical support agent at SupportCo. Write a short, polite
reply to the customer, under 120 words. Suggest one or two simple
things to try, and ask for one missing detail (browser, device or
steps) if you need it.

Ticket: {ticket}
```

`src/prompts/general.txt`

```
You are a friendly support agent at SupportCo. Write a short, polite
reply to the customer, under 80 words. If you don't know a company
```

```
fact, say a team member will follow up. Never make one up.

Ticket: {ticket}
```

Story — `chains/classifier.py`: `build_classifier_chain()` returns the chain (Doc05's Build Task shape: a function that returns a `Runnable`). `classify_ticket()` runs it, catches `ValidationError` by name (Doc05's Intermediate exercise, `extract_person()`), and logs every decision with its reason, like Project 1's router. **If not:** one unreadable answer would crash the whole queue.

```
# src/chains/classifier.py
# The classifier chain. build_classifier_chain() is prompt | structured
# model; classify_ticket() runs it, falls back to "general, not sure" on a
# ValidationError, and logs every decision.
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable
from pydantic import ValidationError

from src.models.llm import get_model
from src.schemas import TicketCategory
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def build_classifier_chain() -> Runnable:
    prompt = ChatPromptTemplate.from_template(load_prompt("classifier"))
    model = get_model().with_structured_output(TicketCategory)
    chain = prompt | model
    return chain

def classify_ticket(ticket: str) -> TicketCategory:
    logger = get_logger(__name__)
    chain = build_classifier_chain()
    try:
        result = chain.invoke({"ticket": ticket})
    except ValidationError:
        logger.warning("Classifier answer did not fit TicketCategory")
        result = TicketCategory(
            category="general",
            reason="The classifier's answer could not be read.",
            sure=False,
        )
    logger.info(
        f"category={result.category} sure={result.sure} "
        f"reason={result.reason}"
```

```
    )
    return result
```

Story — `chains/replies.py`: the branch. The category picks the prompt file, and the rest of the chain is the same for all three. **If not:** three nearly-identical functions, one per category.

```
# src/chains/replies.py
# build_reply_chain(category): one reply chain per category, each loading
# the prompt file named after the category.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable

from src.models.llm import get_model
from src.utils.prompts import load_prompt

def build_reply_chain(category: str) -> Runnable:
    prompt = ChatPromptTemplate.from_template(load_prompt(category))
    chain = prompt | get_model() | StrOutputParser()
    return chain
```

Story — `services/pipeline.py`: the flow — classify, pick, reply — in one function, `handle_ticket()`, that prints the reply and returns a `TicketRecord`. The tests and `main.py` call the exact same code. **If not:** the flow would live inside `main.py`'s loop and nothing else could reuse it.

```
# src/services/pipeline.py
# handle_ticket(ticket, customer_id): classify the ticket, pick the reply
# chain, get the reply, and return a TicketRecord. The one function main.py
# and the tests both call.
from src.chains.classifier import classify_ticket
from src.chains.replies import build_reply_chain
from src.schemas import TicketRecord

def handle_ticket(ticket: str, customer_id: str) -> TicketRecord:
    category = classify_ticket(ticket)
    chain = build_reply_chain(category.category)
    inputs = {"ticket": ticket, "customer_id": customer_id}
    reply = chain.invoke(inputs)
    print(f"Reply: {reply}")

    return TicketRecord(
        customer_id=customer_id,
        ticket=ticket,
        category=category.category,
```

```
        reply=reply,
    )
```

Story — `data/sample/tickets.json` **:** the 4 test tickets, including the unclear one. **If not:** the tickets would be retyped in `main.py` and in every test.

`data/sample/tickets.json`

```
[
  {
    "customer_id": "C-1001",
    "ticket": "I was charged twice for my Pro plan this month (INV-2201).",
    "expected": "billing"
  },
  {
    "customer_id": "C-1002",
    "ticket": "The export button spins forever and never downloads.",
    "expected": "technical"
  },
  {
    "customer_id": "C-1003",
    "ticket": "Do you have a referral program for other companies?",
    "expected": "general"
  },
  {
    "customer_id": "C-1004",
    "ticket": "My invoice and the dashboard numbers don't match. Bug?",
    "expected": "billing or technical"
  }
]
```

Story — `main.py` **:** config first, then every sample ticket through `handle_ticket()` ; `print_record()` shows the result under each ticket. A bad key stops the app; a rate limit skips one ticket (Doc04's error table). **If not:** the first API error would end the run with a traceback.

```
# main.py
# The file you run: python main.py
# Loads the config, reads the sample tickets, runs handle_ticket() on each,
# and prints each result with print_record().
import json
import sys

import openai

from src.config import load_config
```

```
from src.exceptions import MissingConfigError
from src.schemas import TicketRecord
from src.services.pipeline import handle_ticket
from src.utils.logger import get_logger

def load_samples() -> list[dict]:
    with open("data/sample/tickets.json") as samples_file:
        return json.loads(samples_file.read())

def print_record(record: TicketRecord) -> None:
    print(f"  category:      {record.category}")

def main() -> None:
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    samples = load_samples()
    print(f"LangChainPro — {len(samples)} tickets in the queue.")

    for sample in samples:
        print()
        print(f"Ticket from {sample['customer_id']}: {sample['ticket']}")
        try:
            record = handle_ticket(sample["ticket"], sample["customer_id"])
        except openai.AuthenticationError:
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        except openai.RateLimitError:
            print("We're busy right now — this ticket was skipped.")
            continue
        print_record(record)

if __name__ == "__main__":
    main()
```

Also create empty `__init__.py` files in `src/chains/` and `src/services/`.

Back to this step · Hint 1 · Hint 2 · Solution

The Step 1 tests

Story — `tests/unit/test_config.py`: the same three checks as Doc01's `test_config.py`, with no API call, using a made-up setting name so your `.env` is never touched. **If not:** a broken `.env` would first show up as a confusing OpenAI error.

```
# tests/unit/test_config.py
# Checks the settings code with no API call. Each case prints OK or WRONG.
# It only changes settings inside this test run — your .env file is never
# touched.
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "LANGCHAINPRO_TEST_SETTING"

def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting() -> None:
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
```

```
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

Story — `tests/unit/test_schemas.py`: no key needed — it proves an unknown category is rejected and that a record builds from the 4 Step 1 fields. **If not:** a typo category would silently match no branch, and you'd only notice on a real ticket.

```
# tests/unit/test_schemas.py
# Checks the Pydantic shapes accept good data and reject bad data. Each case
# prints OK or WRONG. No API key needed.
# Run from the repo root: python -m tests.unit.test_schemas
from pydantic import ValidationError

from src.schemas import TicketCategory, TicketRecord

def test_good_category() -> None:
    result = TicketCategory(
        category="billing", reason="A double charge.", sure=True
    )
    ok = "OK"
    if result.category != "billing":
        ok = "WRONG"
    print(f"case 1 (good category)   -> {ok}")

def test_unknown_category() -> None:
    ok = "WRONG"
    try:
        TicketCategory(category="shipping", reason="?", sure=True)
    except ValidationError:
        ok = "OK"
    print(f"case 2 (unknown rejected) -> {ok}")

def test_record() -> None:
    record = TicketRecord(
        customer_id="C-1001",
        ticket="I was charged twice this month.",
        category="billing",
        reply="Sorry about that — the billing team will check it.",
    )
    ok = "OK"
    if record.category != "billing":
```

```
        ok = "WRONG"
    print(f"case 3 (record builds)   -> {ok}")

if __name__ == "__main__":
    test_good_category()
    test_unknown_category()
    test_record()
```

Story — `tests/integration/test_live_pipeline.py`: the checks that need the real model — each sample gets the right category, and a billing ticket runs all the way to a reply. Every later step adds its own cases here. **If not:** you'd only know the classifier works on the tickets you happened to try by hand.

```
# tests/integration/test_live_pipeline.py
# Runs the classifier and the whole pipeline against the real API — each
# step adds cases. Each case prints OK or WRONG. Needs OPENAI_API_KEY.
# Run from the repo root: python -m tests.integration.test_live_pipeline
import json

from src.chains.classifier import classify_ticket
from src.config import load_config
from src.services.pipeline import handle_ticket

def load_samples() -> list[dict]:
    with open("data/sample/tickets.json") as samples_file:
        return json.loads(samples_file.read())

def test_billing_classified(samples: list[dict]) -> None:
    result = classify_ticket(samples[0]["ticket"])
    ok = "OK"
    if result.category != "billing":
        ok = "WRONG"
    print(f"case 1 (billing)   -> {ok} ({result.category})")

def test_technical_classified(samples: list[dict]) -> None:
    result = classify_ticket(samples[1]["ticket"])
    ok = "OK"
    if result.category != "technical":
        ok = "WRONG"
    print(f"case 2 (technical) -> {ok} ({result.category})")

def test_general_classified(samples: list[dict]) -> None:
    result = classify_ticket(samples[2]["ticket"])
    ok = "OK"
    if result.category != "general":
        ok = "WRONG"
```

```
    print(f"case 3 (general)   -> {ok} ({result.category})")

def test_full_pipeline(samples: list[dict]) -> None:
    record = handle_ticket(samples[0]["ticket"], samples[0]["customer_id"])
    ok = "OK"
    if record.category != "billing" or record.reply == "":
        ok = "WRONG"
    print(f"case 4 (full pipeline) -> {ok}")

if __name__ == "__main__":
    load_config()
    samples = load_samples()
    test_billing_classified(samples)
    test_technical_classified(samples)
    test_general_classified(samples)
    test_full_pipeline(samples)
```

Run it (from the repo root):

```
python -m tests.unit.test_config
python -m tests.unit.test_schemas
python main.py
python -m tests.integration.test_live_pipeline
```

Expected unit output:

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (good category)   -> OK
case 2 (unknown rejected) -> OK
case 3 (record builds)   -> OK
```

Look closely at the unclear ticket (`C-1004`): which category did it get, and did the model say `sure=False` ? Nothing acts on `sure` yet — Step 4 does.